

Programming Exercise: Mini-Batch Stochastic Gradient Descent

Foundation of Data Science
Shiraz University

1 Objective

In class, you learned about two extremes: Batch Gradient Descent (GD) which uses all samples, and Stochastic Gradient Descent (SGD) which uses one sample at a time. In this assignment, you will explore **Mini-batch SGD**, which sits between these extremes and is the most commonly used variant in practice. You will implement mini-batch SGD and systematically compare its performance with both GD and SGD to understand the trade-offs.

2 Dataset: NYC Taxi Trip Data

The dataset used in this assignment comes from the NYC Taxi & Limousine Commission (TLC) and consists of yellow taxi trip records stored in Parquet format. The predictive task is to estimate trip duration in seconds based on features such as pickup time, trip distance, passenger count, and location IDs. This dataset is particularly well-chosen for the exercise because it is sufficiently large (e.g., the January 2024 file contains nearly 2.9 million records) to highlight the practical differences in performance and convergence behavior across gradient descent variants, while still being manageable on a standard personal computer when appropriately subsampled.

2.1 Data Access

```
# In your terminal or Jupyter notebook, run:  
pip install pandas numpy scipy matplotlib scikit-learn pyarrow
```

1. Visit: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
2. Download: `yellow_tripdata_2024-01.parquet` (January 2024, ~54MB)
3. Save to your working directory

```
import pandas as pd  
  
# IMPORTANT: Restart your kernel after installing pyarrow!  
# Load the parquet file  
df = pd.read_parquet('yellow_tripdata_2024-01.parquet')  
print(f"Loaded{len(df)}rows")
```

2.2 Recommended Dataset Sizes

Test with multiple sizes to understand scaling behavior:

- **Small (100K rows):** Debug your code, verify correctness (fast iteration)
- **Medium (500K rows):** Observe clear performance differences between algorithms
- **Large (full dataset):** Experience real-world trade-offs in time vs. convergence

2.3 Data Preprocessing

```
import pandas as pd
import numpy as np

# 1. Load data
df = pd.read_parquet('yellow_tripdata_2024-01.parquet')

# 2. Create target variable (trip duration in seconds)
df['trip_duration'] = (df['tpep_dropoff_datetime'] -
df['tpep_pickup_datetime']).dt.total_seconds()

# 3. Extract time features
df['hour'] = df['tpep_pickup_datetime'].dt.hour
df['day_of_week'] = df['tpep_pickup_datetime'].dt.dayofweek

# 4. Filter outliers (trips between 1 min and 2 hours)
df = df[(df['trip_duration'] >= 60) & (df['trip_duration'] <= 7200)]
df = df[df['trip_distance'] > 0]

# 5. Select features
features = ['trip_distance', 'passenger_count', 'hour',
'day_of_week', 'PULocationID', 'DOLocationID']
X = df[features].values
y = df['trip_duration'].values

# 6. Normalize features (IMPORTANT!)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 7. Subset for testing (start small!)
X = X[:50000]
y = y[:50000]
```

3 Part 1: Mini-Batch SGD Implementation

3.1 Understanding Mini-Batch SGD

Before implementing, answer these questions in your report:

1. What is the key difference between batch GD, SGD, and mini-batch SGD?
2. Why might mini-batch SGD combine the advantages of both GD and SGD?
3. What happens to mini-batch SGD when batch size = 1? When batch size = n (dataset size)?

3.2 Implementation Requirements

Implement mini-batch SGD for linear regression. Your implementation should:

```
def minibatch_sgd(X, y, batch_size, learning_rate, n_epochs):  
    """  
    Mini-batch Stochastic Gradient Descent for Linear Regression  
  
    Parameters:  
    -----  
    X : array, shape (n_samples, n_features)  
    Training data  
    y : array, shape (n_samples,)  
    Target values  
    batch_size : int  
    Number of samples per gradient update  
    learning_rate : float  
    Step size for updates  
    n_epochs : int  
    Number of passes through the dataset  
  
    Returns:  
    -----  
    weights : array, shape (n_features,)  
    Learned weights  
    cost_history : list  
    Cost after each mini-batch update  
    time_history : list  
    Cumulative time after each mini-batch update  
    """  
    # Your implementation here  
    pass
```

Key implementation details:

- Shuffle the data at the beginning of each epoch
- Process data in batches of size `batch_size`
- Handle the last batch (which might be smaller than `batch_size`)
- Track cost and time for analysis
- Use vectorized operations for efficiency

3.3 Testing Your Implementation (10 points)

Test your mini-batch SGD with different batch sizes:

- Batch size = 1 (equivalent to SGD)
- Batch size = 32 (small mini-batch)
- Batch size = 128 (medium mini-batch)
- Batch size = 512 (large mini-batch)
- Batch size = n (equivalent to batch GD)

Verify that:

- Batch size = 1 gives similar results to the SGD code from class
- Batch size = n gives similar results to the GD code from class

4 Part 2: Comprehensive Comparison

4.1 Convergence Behavior

1. Create a convergence plot showing cost vs. time for:

- Batch GD (from class)
- SGD (from class)
- Mini-batch SGD with batch sizes: 32, 128, 512

2. Answer:

- Which converges fastest to a good solution?
- Which has the smoothest convergence curve? Why?
- What happens to the convergence behavior as batch size increases?

4.2 Computational Efficiency

3. Create a comprehensive comparison table for at least 2 dataset sizes (e.g., 200K and 500K rows):

Algorithm	Time/Epoch	Final Cost
Batch GD		
SGD (batch=1)		
Mini-batch (32)		
Mini-batch (128)		
Mini-batch (512)		

4.3 Effect of Batch Size

4. Plot: **Final cost vs. Batch size** after 10 epochs.

5 Part 3: Critical Analysis

5.1 The Batch Size Dilemma

Write 2-3 paragraphs addressing:

- Based on your experiments, why don't we always use very large batch sizes?
- Based on your experiments, why don't we always use very small batch sizes?

5.2 Learning Rate and Batch Size Relationship

1. Experiment: Test the same learning rate (e.g., 0.01) with different batch sizes
 - Does the same learning rate work well for all batch sizes?
 - What happens when you use SGD's optimal learning rate with batch GD?
2. Write a paragraph explaining the relationship between batch size and learning rate based on your findings

6 Tips

- Fix the random seed to ensure reproducibility of your results. This is especially important when shuffling data. In Python, you can set seeds for NumPy and other libraries as follows:

```
import numpy as np
import random

np.random.seed(42)
random.seed(42)
```

- When comparing algorithms (GD, SGD, mini-batch SGD), use the **same initial weights** to ensure a fair comparison.
- Monitor both **training cost** and **wall-clock time**. Convergence in iteration count does not always translate to faster real-world performance.